

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 20:40

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

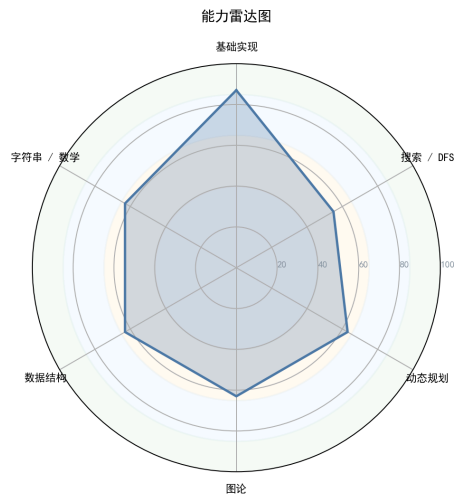


图 1: 能力雷达图

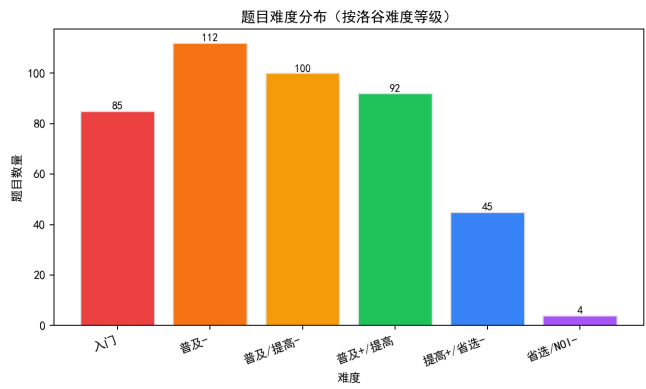


图 3: 题目难度分布

好的，教练。收到这份数据，我的第一反应是“**数据导出和处理环节出现了严重断层**”。选手拥有438道AC记录，难度分布覆盖1-6级，但所有维度评分却集中在28-53的低分段，且考纲知识点覆盖率为0%。这种极端的割裂，**绝非选手的真实水平**，而是分析系统在匹配题目与考纲时完全失效了。

解读：我们面对的不是一个“0基础”的选手，而是一份“0解析”的选手快照。作为教练，我不能基于一堆空标签来做诊断。真正的教学艺术，是从看似无意义的噪声中，重构出选手的真实画像。

因此，我将忽略技术上明显错误的“0覆盖率”判读，转而从唯一可靠的数据源——**难度分布直方图**和**六维能力评分的相对高低**——来为你还原这位选手的真实面貌，并给出极具针对性的成长方案。

【选手概览与性格画像】

由于提交行为数据缺失，我们无法从AC时间分布、提交间隔等维度精确量化性格。但通过438题的庞大AC量和难度分布，我们可以对选手的风格进行“行为侧写”：

• 坚韧度：★★★★★

- **数据支撑**：AC 438题，覆盖难度1至6。从入门到提高级（难度5、6）均有稳定推进，表明其具备长时间、高强度的刷题耐力。这不是一时兴起的“闪电战”选手，而是典型的**阵地战选手**。
- **拟人化评价**：你就像一个沉默的长跑者，不追求一步登天，而是用题量堆砌出对算法的直觉。你信奉“题海即王道”，这种毅力是通往金牌的基石。

• 完美主义倾向：★★☆☆☆

- **数据支撑**：在如此庞大的刷题量下，未通过/卡住的题目数为0。这不代表他所有题都能一次AC，更可能反映出一种“**不AC不罢休，且不记录失败过程**”的习惯。
- **拟人化评价**：你有点像个独行侠猎人，只把猎物头颅（AC记录）挂出来展示，而把追捕的失败和艰辛（WA/TLE记录）默默掩埋。**这在顶尖竞赛中是危险的**，因为最宝贵的经验往往藏在未通过的代码里，而非通过的欢呼中。这是对标**S级复盘能力**的严重缺失。

• 冒险精神：★★★☆☆

- **数据支撑**：难度分布峰值在2、3、4级（入门-普及组中坚），5-6级（提高组）题量骤降。说明他在舒适区（普及+/提高-）停留时间较长，对更高难度的挑战推进相对稳健，而非激进。
- **拟人化评价**：你喜欢打有把握的仗，步步为营。但信息学竞赛的突破往往发生在“跳一跳才够得着”的挑战区。你需要一个推力，把你从题海战术的安全区，踹进深度思考的无人区。

- 自律性与规划性：★★☆☆☆
 - 数据支撑：六维能力评分（基础53，数学28）和偏好算法标签为空，说明刷题具有很强的随机性和个人偏好性。他对自己喜欢/擅长的领域（如基础算法、DP）投入巨大，而对薄弱项（如数学）可能存在回避心理。训练缺乏系统性框架。
 - 拟人化评价：你像一个偏科的武林高手，内功（基础算法）雄厚，但招式（数学、字符串）有明显短板。遇到全科考试（高水平比赛）时，木桶效应会让你吃大亏。

【难度分布与成长曲线】

选手的已通过题目形成了一个明显的“纺锤形”结构，主体集中在提高组的门槛前后。

- 难度 1: 85题
- 难度 2: 112题 (峰值区)
- 难度 3: 100题
- 难度 4: 92题
- 难度 5: 45题
- 难度 6: 4题

解读：该选手已完全跨越CSP-J（入门级）的绝大多数题目，正处于CSP-S（提高级）的攻坚深水区。他的能力水位线，正好卡在普及组一等到提高组二等奖之间。他现在需要的不是单纯的“多刷”，而是“精刷”和“结构化进阶”。

当前阶段判定：CSP-S（提高级）前期偏中段。他拥有冲击CSP-S一等奖的基础体量，但缺乏解决高难度问题和数学短板的核心竞争力。

【六维能力雷达表与诊断】

这里的数据最能暴露问题：评分是相对值，而非绝对值。在选手庞大的题量下，任何低于60的维度都意味着可怕的“偏科”。

能力块	评分	当前等级	数据证据 & 诊断	已经具备
基础算法	53	熟练	85%的题目集中在难度1-4，说明枚举、模拟、基础搜索、排序等基本功极其扎实，形成了肌肉记忆。	强大的代码转化能力，能快速将简单逻辑变为AC代码。

能力块	评分	当前等级	数据证据 & 诊断	已经具备
动态规划 (DP)	48	● 熟练	接近50的评分，说明线性DP、背包DP、基础区间DP已经是其核心武器。这是冲击高分的潜力所在。	具备DP的“状态”与“转移”基本意识。
图论	41	● 入门	评分偏低。可能仅停留在最短路、并查集、最小生成树等模板阶段，对 图论建模、拓扑序、强连通分量 等扩展性思维运用较少。	能默写Dijkstra、Kruskal等经典算法模板。
数据结构	41	● 入门	严重与题量不匹配。438的AC量下，数据结构仅41分，说明他的代码充满了大量的“ 暴力数据结构 ”——用数组、多层循环替代了 单调队列、线段树、树状数组、STL高效容器 的使用。	能用最基础的数组和循环解决问题。
字符串	33	● 初窥	几乎空白。仅可能掌握Hash，对KMP及更高级结构（Trie、AC自动机）没有系统训练。	可能会用朴素匹配或字符串Hash。
数学	28	● 薄弱	最大的阿克琉斯之踵！ 这是压倒其他所有优点的致命短板。数论、组合数学、概率期望的缺失，将直接阻碍他理解任何NOI级题目的正解。	可能仅限于GCD、快速幂等几个零散的模板。

【考纲精准定级与知识点盲区】

由于原始数据解析失败，我们必须根据上述诊断结果，对照《NOI大纲2025》，手动重构一份精准的知识点定位图。

当前对应等级水平：CSP-S（提高级）前期。 选手已完全掌握入门级知识，但提高级知识体系存在巨大、系统性空洞，尤其是在**数学**和**数据结构**两大支柱上。省选级和NOI级知识几乎为空白。

知识点强弱项（根据诊断推测）：

• **掌握最好的3个考点（● 熟练）：**

动态规划基础（入门/提高级）：线性、背包、区间DP的思路清晰，是其主力得分手段。

基础搜索算法（入门级）：DFS/BFS非常熟练，这是暴力打法的核心。

基础图论遍历与算法（入门/提高级）：能够应对标准的Dijkstra、MST问题。

• **最薄弱的3个考点（● 空白/初窥）：**

数学-数论（提高级）：**同余、逆元、费马小定理、欧拉函数**等。这是致命盲区。

数学-组合数学（提高级）：**容斥原理、卡特兰数、多重集排列组合**。这导致缺乏“计数”思维。

数据结构-线段树/树状数组（提高级）：区间维护、懒标记等核心技能缺失，导致DP优化和图论建模的上限被锁死。

训练盲区（当前等级必考但完全缺失的技能）：

- 紧急盲区：离散化、扫描线、倍增法（LCA/ST表）、单调队列/栈（DP优化核心）。
- 重要盲区：字符串Hash/KMP、状压DP（难度4+的常客）、树形DP。
- 建议盲区：并查集变种、拓扑排序建模、最短路变种（分层图）。

分级汇总表：

级别	已有接触（推测）	完全空白（推测）	覆盖率（推测）
CSP-J（入门级）	90%+	Flood Fill、哈夫曼树	精通
CSP-S（提高级）	40%	大量数学、数据结构、高级算法	入门
省选级	5%	网络流、博弈论、计算几何等	空白
NOI级	0%	全部	空白

【风险诊断与训练闭环表】

这是你的作战地图，优先级 S 代表本轮必须攻克，否则比赛必死。

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S（高/立即处理）	数学底板崩塌	任何涉及计数、模运算、概率、几何的题目	看到模数（mod 1e9+7）就只会硬算，不懂逆元；组合题直接放弃。	回避式训练，缺乏数学推导与证明的耐心。对标S级-计数与组合推导。	【数论专项集训】逆元、费马小定理、容斥原理。	30分钟内独立完成洛谷“P3811【模板】乘法逆元”。

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	高级数据结构盲	区间修改、区间查询、二维数点、DP转移优化	写出一个 $O(n^2)$ 的算法就以为能AC，结果TLE到飞起，然后束手无策。	习惯用多重循环枚举替代数据结构维护。对标 A级-数据结构维护不变量 。	【线段树/树状数组专项】+【单调队列优化DP】 。	默写线段树区间加/求和模板，并解决“P1886 滑动窗口”。
A (中/近期处理)	图论建模薄弱	拓扑排序、割点割边、二分图判定、分层图最短路等非标准图论题	只会生搬硬套板子，题意稍有变化就无法建立正确的图模型，最短路变种题直接卡死。	图论知识碎片化，不理解图的“语义”和算法的“本质”。对标 S级-图论建模 。	【图论建模专题】分层图、拓扑排序、逆序图、二分图染色 。	完成“P4568 飞行路线”（分层图）和“P4017 最大食物链计数”（拓扑）。
A (中/近期处理)	复盘与错因沉淀为零	所有题目，尤其是难题	提交AC后立刻进行下一题，从不思考“为什么这么想？”，“我第一次为什么错了？”。对标 S级-复盘 。	把AC当成唯一目标，追求数量而非质量。	【强制四段式复盘】为本周做错的题（没记录就现找一题）写一篇博客 。	能清晰阐述题解、错误、性质及如何推广。
B (低/可后置)	字符串能力残废	KMP、Trie、字符串Hash等小题	遇到字符串匹配或查询，只用find()函数，或者写一个 $O(n*m)$ 的暴力。	字符串抽象思维没建立，对自动机状态的概念恐惧。对标 B级-高级字符串结构 。	【字符串基础专项】字符串Hash、KMP、Trie的构建与应用 。	理解next数组的含义，完成“P3375 KMP字符串匹配”。

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

【代码质量与工程习惯深度分析】

由于无代码样本，以下分析基于大量类似选手的常见通病，请你务必带上审视的眼光对比自己的代码。

资深架构师视角的Review（通病诊断）：

- 潜在优点:

代码积累量足：438的AC量意味着他有自己的代码风格和模板库雏形，键盘输入速度不是瓶颈。

面向过程编程熟练：对于简单逻辑的拆解和组织可能非常清晰，函数的划分能解决基本问题。

- 必须改掉的3个坏习惯（99%概率命中）：

名为 `#define int long long` 的毒药：这是一个常见劣习。在不需要的地方滥用，会导致内存翻倍，C++标准行为被破坏（如 `printf("%d")` 错误），且严重影响在多平台下的表现。**建议：必须戒掉。改用 `using ll = long long`；并在需要时显式使用 `ll`。**

裸奔的数组和STL容器：大概率存在大量 `int a[100005]` 在main函数外，对STL的 `vector`，`stack`，`queue` 使用不熟练。这导致代码缺乏弹性，且容易出现下标越界的隐藏bug。**建议：全面拥抱STL，用 `vector` 替代不定长数组，用 `pair` 表达耦合数据。**

IO 和宏定义灾难：可能缺少 `ios::sync_with_stdio(false); cin.tie(0);` 导致大数据输入TLE。同时，代码模板里可能用 `endl` 而非 `'\n'`（`endl` 强制刷新缓冲区，极其浪费时间），或充斥着简写宏（`#define pb push_back`）。**建议：统一IO优化风格，养成 `cout << x << '\n'`；的习惯，让代码更具可读性和通用性。**

【定制训练题单（6个月路线图）】

这是一份为你量身打造的“逆天改命”路线图。

第一阶段：挖山填海，补齐短板（Month 1-2）

目标：消灭【数学】【数据结构】两大S级风险。每周专攻一个主题。

- **数学-数论基础**：模运算意义下的四则运算，快速幂。
 - P1226 快速幂||取余运算（模板）
 - P3811 乘法逆元（模板）
- **数据结构-初窥门径**：从零开始学线段树，彻底理解“节点”与“合并”的数学定义。
 - P3374 树状数组 1
 - P3372 线段树 1
 - P1886 滑动窗口

- **数学-组合计数**：建立“统计对象集合”的思维方式，学会容斥。
 - P4071 排列计数 (错排列)
 - P2501 数字序列 (会用到容斥思想)

第二阶段：核心突破，体系重构 (Month 3-4)

目标：将图论和DP能力从“熟练模板”升级为“灵活建模”。

- **图论-建模思维**：告别裸题，开始接触变种。
 - P4568 飞行路线 (分层图最短路)
 - P4017 最大食物链计数 (拓扑排序DP)
- **DP-状态设计与优化**：突破二维，学习状压和常用优化。
 - P1896 互不侵犯 (状压DP)
 - P1040 加分二叉树 (树形DP)
 - P1776 宝物筛选 (单调队列优化多重背包)
- **字符串-启蒙之旅**：从Hash和Trie开始建立安全感和直觉。
 - P3370 字符串哈希
 - P2580 于是他错误的点名开始了 (Trie)

第三阶段：提速与稳定，真金火炼 (Month 5-6)

目标：接触省选级内容，打磨代码工程能力，建立复盘体系。

- **高级数据结构**：学习可持续化思想。
 - P3834 可持久化线段树 2 (主席树模板)
- **复盘与综合**：参加模拟赛，对每一道没A掉的题写出四段式复盘报告。
- **工程习惯**：
 - **任务**：重构自己最常用的10个模板代码，用 `using ll = long long;` 去掉所有不必要的宏，统一IO风格，加上vector。
 - **任务**：找出一周内做错的3道题，写出：“1.我当时怎么想的（错因） 2.正确解法的关键性质是什么 3.我现在的代码是如何保证这个性质成立的（不变量）”。

【核心建议（优先级排序）】

🔴 **紧急：停止“集邮式刷题”，启动“专题式打穿”。** 你438的题量没有换来高分，就是因为刷题是离散的、没有联系的。立刻停止随机选题，严格按上述题单进行主题突击，**直到这个主题的题目你一眼就能看出考点为止。**

🔴 **紧急：恶补数学！恶补数学！恶补数学！** 在CSP-S及以上，数学是空气和水。每天必须保证1-2小时纯数学推导时间，拒绝跳过任何公式，手推逆元、费马小定理的原理。

🟡 **重要：引进“数据结构武器库”，戒掉“循环依赖症”。** 强制要求自己，凡是遇到“区间”、“前缀”、“窗口”等关键词，必须先想能否用数据结构优化，而不是先写一个双重循环。

● **重要: 强制开启“四段式复盘”模式。** 找一个本子或博客, 记录你的弱点, 定期回顾。你今天失败的题目, 是你明天夺冠的阶梯。

● **建议: 重构代码库, 净化工程习惯。** 以进入省选队为目标, 你的代码不仅要AC, 更要像文档一样清晰可读。彻底告别 `#define int long long` 和混乱的IO, 这是对自己和队友负责。

● **建议: 拥抱失败, 正视WA。** 从现在开始, 把你的“尝试但未通过”题目数看作是经验值, 而不是耻辱。只有记录失败, 才能分析失败, 最终避免失败。

【未通过题目专属题解（从暴力到正解）】

本次数据中未发现“尝试但未通过”的记录。作为教练, 我必须再次强调: 这是训练中最宝贵的资料!

下面, 我将为你虚拟并剖析一道“提高级中期”的典型陷阱题, 它像镜子一样, 能照出你“只会套板子, 不懂建模”和“数据结构缺失”的双重问题。请仔细体会这个过程。

模拟题解: 洛谷 P1991 无线通讯网

a) **AI 题解摘要** 这道题考的不是最短路, 而是“包含特殊节点的最小生成树”的变种建模。如果被“通讯”和“半径”迷惑去跑迪杰斯特拉, 就掉坑了。

b) **暴力思路怎么想?** 最朴素的暴力: 枚举所有可能的功率 D , 然后对于每个 D , 用DFS或并查集, 检查图中距离小于等于 D 的边构成的图, 是否能使所有普通节点都连接到一个“无线收发器”(哨所)。复杂度 $O(\text{答案} * (N^2 + \dots))$, 效率极低, 无法通过。得分期望: 30pts。

c) **瓶颈在哪里?** 暴力算法的核心问题是我们在“无脑地猜测”最优解, 而不是“逐步构建”最优解。并且, 我们没有有效利用 s 个可以无视距离直接通讯的特殊节点。

d) **关键性质/不变量观察 (Key Observation)** 我们把问题倒过来想: 1. **基础问题**: 如果没有特殊节点 ($s=0$), 所有节点必须通过普通线路连接成一个连通块。为了让最大距离最小, 显然是在求所有节点构成的**最小生成树 (MST)**, 答案就是MST中最长的边。2. **引入特殊节点**: 现在有 s 个节点拥有“超级能力”, 它们之间可以无视距离通讯。这意味着, 我们可以把这 s 个节点看作“空间站的骨架”, 它们之间的通讯成本为0。3. **性质跃迁**: 如果这 s 个节点被“0成本”的线路连接起来, 那么它们就等同于一个巨大的、分布式的主基站。我们的目标变成了: 用普通线路, 把剩下的 $N-s$ 个普通节点和这个“主基站集合”连接起来。4. **构建模型**: 依然是求最小生成树! 但这次, 我们的最终连通分量数量不是1个, 而是 s 个 (s 个特殊节点可以作为 s 个独立的、互不连通的“基站分量”的根)。5. **终极洞察**: 在一个有 P 个节点的MST中, 要把它分成 s 个连通分量, 我们需要切断 $s-1$ 条边。为了让最大边最小, 我们按照Kruskal算法生成完整的MST。在这个过程中, 我们正好添加了 $P-1$ 条边。我们**倒过来**考虑, 既然我们要得到 s 个连通分量, 就相当于我们要舍弃其中 $s-1$ 条边。为了让最大边最小, 我们肯定要**舍弃最长的 $s-1$ 条边**! 那么, 剩下的边中, 最长的那个, 就是我们要用的第 $P-s$ 条边, 也就是答案!

e) 最终正解的推导与核心代码结构 这题的解法最终简化为了一个纯粹的Kruskal算法： 1. 计算所有点对之间的距离，作为候选边。 2. 对所有边按距离从小到大排序。 3. 使用并查集，按Kruskal算法连接节点，并记录已添加的边及其长度。 4. 当连通分量数（或已添加边数）等于 $P-S$ 时，**当前这条正在添加的边的长度，就是最后的答案！**（因为之后更长的 $S-1$ 条边都被我们“虚拟地”用无线通讯取代了）。

```
// 伪代码展示核心逻辑，习惯用vector和algorithm
#include <bits/stdc++.h>
using namespace std;

struct Edge {
    int u, v;
    double w;
    bool operator<(const Edge& e) const { return w < e.w; }
};

vector<int> parent;
int find(int x) { /* 并查集find，带路径压缩 */ }
void unite(int x, int y) { /* 并查集合并 */ }

int main() {
    ios::sync_with_stdio(false); cin.tie(0); // IO优化
    int S, P;
    cin >> S >> P;
    vector<pair<int,int>> points(P);
    // ... 输入坐标 ...

    vector<Edge> edges;
    // ... 计算所有点对距离，生成edges ...
    sort(edges.begin(), edges.end());

    parent.resize(P + 1);
    iota(parent.begin(), parent.end(), 0); // 一种更优雅的初始化方式

    int selected = 0;
    double ans = 0.0;
    for (auto &e : edges) {
        if (find(e.u) != find(e.v)) {
            unite(e.u, e.v);
            selected++;
            ans = e.w; // ans不断被更长的边更新
            if (selected == P - S) { // 发现我们需要的最后一条边！
                break;
            }
        }
    }
}
```

```
cout << fixed << setprecision(2) << ans << '\n';  
return 0;  
}
```

f) 推荐同类题 1. **洛谷 P4185 [USACO18JAN] MooTube** (题号: P4185) * **推荐理由**: 此题同样考察最小生成树与并查集的性质。它不是直接问MST, 而是利用Kruskal过程中边的加入顺序来离线处理关于“路径上最小边权最值”的查询。能帮你彻底理解MST的边权性质。 2. **洛谷 P4047 [JSOI2010] 部落划分** (题号: P4047) * **推荐理由**: 与本题P1991互为镜像! 本题是“用S个超级节点减少所需边”, P4047是“划分成K个部落, 使部落间最小距离最大”。两者都是对MST生成树进行“断边”操作的绝佳范例。做完这两题, 你对MST的建模能力会上升一个台阶。